

ApplyFlow Comprehensive Documentation

1. Introduction

ApplyFlow is an enterprise-grade career management and automation platform. It is designed to act as a personal CRM for job seekers, combining ATS resume building, automated email outreach, job discovery, and pipeline analytics.

2. System Architecture

ApplyFlow follows a modern client-server architecture.

Frontend

- **Framework:** Next.js (React)
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **State Management:** Redux Toolkit for global state, React Query for server state.
- **Routing:** Next.js App Router.

Backend

- **Framework:** Express.js (Node.js)
- **Database:** MongoDB (accessed via Mongoose)
- **Authentication:** JWT-based session management.
- **Email Automation:** Nodemailer combined with agenda/cron for scheduling.

3. Data Models & Schemas

User

Stores user authentication details, profile settings, and subscription status.

- `email` (String, Unique)
- `passwordHash` (String)
- `profileData` (Object)

Resume

Stores the JSON representation of ATS resumes.

- `userId` (ObjectId)
- `title` (String)
- `content` (Object - contains work history, education, skills)

- `templateId` (String)

Job Application (Pipeline)

Tracks the status of applied jobs.

- `userId` (ObjectId)
- `companyName` (String)
- `role` (String)
- `status` (Enum: Saved, Applied, Interviewing, Offer, Rejected)
- `appliedDate` (Date)

Automation Workflow

Defines custom logic for email sequences and pipeline updates.

- `userId` (ObjectId)
- `trigger` (String)
- `actions` (Array of action objects)

4. How to Build and Run Locally

Prerequisites

- Node.js (v16+)
- MongoDB running locally or a MongoDB Atlas URI
- Git

Backend Setup

1. Open a terminal and navigate to the backend directory:

```
cd ApplyFlow-Backend
```

2. Install dependencies:

```
npm install
```

3. Configure environment variables in `.env`:

```
PORT=5000  
MONGODB_URI=mongodb://localhost:27017/applyflow  
JWT_SECRET=your_super_secret_key
```

4. Start the backend:

```
npm run dev
```

Frontend Setup

1. Open a new terminal and navigate to the frontend directory:

```
cd ApplyFlow-Frontend
```

2. Install dependencies:

```
npm install
```

3. Configure environment variables in `.env`:

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
```

4. Start the frontend server:

```
npm run dev
```

5. Open your browser and navigate to `http://localhost:3000`.

5. Development Workflow & Contribution

When adding a new feature:

1. **Create a branch:** `git checkout -b feature/your-feature-name`
2. **Backend implementation:** Create the necessary Mongoose models, controllers, and routes. Ensure you write unit tests for your logic.
3. **Frontend implementation:** Build the React components. Use Tailwind for styling. Fetch data using React Query hooks.
4. **Testing:** Run `npm run test` in both directories to ensure nothing is broken.
5. **Commit:** Write descriptive commit messages.
6. **Pull Request:** Push your branch and open a PR against the `main` branch.

6. Deployment Guide

ApplyFlow is designed to be easily deployed to modern cloud providers.

Deploying the Backend

- Recommended: Render, Heroku, or AWS Elastic Beanstalk.
- Ensure you set all production environment variables (e.g., `NODE_ENV=production`).
- Ensure your MongoDB Atlas cluster allows incoming connections from your backend IP.

Deploying the Frontend

- Recommended: Vercel or Netlify.
- Connect your GitHub repository to Vercel.
- Set the `NEXT_PUBLIC_API_URL` to your live backend URL (e.g.,

`https://api.yourdomain.com/api).`

- Vercel will automatically build and deploy your Next.js application.

Documentation generated automatically by the ApplyFlow core team.